

Practical SNARGs for Matrix Multiplications over Encrypted Data

Louis Tremblay Thibault
Zama and École de technologie supérieure
louis.tremblay.thibault@zama.ai

Michael Walter
Zama
michael.walter@zama.ai

Jiapeng Zhang
University of Southern California
jiapengz@usc.edu

Abstract—Fully Homomorphic Encryption (FHE) enables computations to be performed directly on encrypted data, without ever requiring decryption. This capability is particularly crucial for privacy-preserving outsourced computation in sensitive fields such as healthcare and finance. While FHE ensures data confidentiality under the honest-but-curious adversarial model, achieving full malicious security, encompassing both integrity and privacy, requires an additional layer of verifiability.

To address this, a growing body of research has explored combining FHE with techniques from verifiable computation, leading to the notion of verifiable FHE (vFHE). However, the integration of these two paradigms often results in substantial computational overhead, making existing approaches largely impractical for real-world deployment.

In this work, rather than targeting general-purpose verifiable FHE, we design a novel and practical verifiable homomorphic encryption scheme tailored for an important and widely used operation: matrix–vector multiplication. We provide an open-source implementation and our experimental results demonstrate that the proposed scheme achieves high efficiency, making it ready for practical adoption.

Index Terms—SNARGs, Verifiable Homomorphic Computation, Privacy-preserving Machine Learning

1. Introduction

Fully Homomorphic Encryption (FHE) is powered by its ability to perform computations directly on encrypted data, thereby eliminating the need for decryption during the computation. This property is highly desirable for privacy-preserving outsourced computation, especially over data containing sensitive information. While FHE ensures confidentiality against honest-but-curious adversaries, achieving full malicious security, which requires both integrity and privacy, demands an additional verifiability layer. Consequently, recent research has increasingly explored the combination of FHE with verifiable computation techniques, leading to the emergence of verifiable Fully Homomorphic Encryption (vFHE) [1]–[9].

The main framework of vFHE is that, in addition to returning the ciphertext of the computational outcome, the server is also required to generate a proof demonstrating that the ciphertext was obtained through an honest execution of the designated homomorphic computation.

In current vFHE constructions, the main tool used to achieve efficient verifiability is the *Succinct Non-Interactive Argument* (SNARG). Although this problem has been extensively studied, integrating these two powerful paradigms still incurs significant computational overhead, rendering existing solutions largely impractical for real-world deployment.

To make a verifiable homomorphic computation practical, several factors are in an important consideration:

- 1) The server’s running time, including both homomorphic computation and proof generation, should be efficient in practice.
- 2) The proof size should be compact.
- 3) Most importantly, the client’s running time, including encryption, decryption and verification, should be less than running the computation over plaintext by themselves. Otherwise, there is no incentive to use FHE.

A large body of prior work has focused on optimizing the proving time of general FHE schemes (e.g., [3], [4], [6], [7]). However, we do not see many works that compare their verification time with the plaintext computation time. See also [10].

In fact, it is very challenging to achieve all three of these points for general fully homomorphic computation. Rather than aiming for a general but impractical vFHE scheme, we focus on the following question:

Can we realize verifiable homomorphic computation for some specific yet meaningful computation?

In this paper, we study a meaningful and widely used computation: matrix–vector multiplication, and present a new practical verifiable homomorphic encryption scheme for it. Our scheme allows to answer this question by the affirmative.

Matrix-vector multiplication. Let $\mathbf{A} \in \mathbb{Z}_l^{n \times m}$ and $\mathbf{x} \in \mathbb{Z}_l^m$, where $\mathbf{A}$ is public and $\mathbf{x}$ is private and $l \in \mathbb{N}$. The goal is to securely outsource the computation of $\mathbf{y} \leftarrow \mathbf{Ax}$. In this direction, we emphasize that matrix–vector multiplication is one of the most widely used arithmetic computations, with numerous applications, particularly in machine learning. For example, many fundamental machine learning algorithms, including PCA, linear regression, and SVMs, rely heavily on repeated matrix–vector multiplications as their core computational primitive. Even in large language models (LLMs), matrix–vector multiplication plays a crucial role.

As a consequence, privacy-preserving matrix–vector multiplication has also been extensively studied in prior works [11]–[15]. Noticeably, our work is also inspired by [16], which introduces an interactive protocol for private LoRA (Low-Rank Adaption) fine-tuning of (public) LLMs. The authors remark that the protocol is also useful for inference on private query vectors. The key idea of their protocol is to outsource only the heavy linear computation of the fine-tuning and process the lighter non-linear part locally. The linear computation, i.e. the multiplication of a public matrix and a private vector, is performed using an additively homomorphic encryption scheme. A major weakness of the system is the amount of trust that must be put into the server. More precisely, the protocol needs to assume that the server is at most honest-but-curious, meaning that it follows the protocol honestly but may attempt to learn information from the protocol execution. Under this assumption, correctness is obvious and security of the interactive protocol follows from the work of [17], since it is *cleartext computable*. Adding verifiability to the outsourced matrix-vector multiplication protocol provides correctness and privacy in a stronger security model, where the server may be fully malicious.

1.1. Previous Challenges and Our Solution

Homomorphic multiplication for matrices and vectors may appear to be a simple and solved problem at first glance, as it involves only linear operations, and there are several existing works on privacy-preserving matrix multiplication. However, to make it applicable to broader applications such as blockchains, public verifiability with sublinear verification time is crucial. Previously, sublinear verification time has typically been achieved using standard, well-established approaches that generate SNARG proofs for the homomorphic computation. We note that the majority of current SNARG techniques are built over finite fields.

However, to ensure efficient homomorphic computation, Ring-LWE is generally a more practical assumption than plain LWE. In this regime, all homomorphic computations are carried out over polynomial rings. Proving the correct execution of ring arithmetic using only finite fields is usually very inefficient, which constitutes a major technical obstacle to practicality.

To resolve this challenge, there are two main existing approaches. The first direction mainly follows the Rinocchio paper [18]. In this direction, Rinocchio introduced Quadratic Ring Programs (QRPs), an adaptation of Quadratic Arithmetic Programs (QAPs) to the ring setting. It aims to prove the correctness of homomorphic computation directly over rings via QRPs. However, it has two main drawbacks:

- It requires a trusted setup, i.e., it is not publicly verifiable. Such a limitation is not favorable for blockchain-related applications.
- The exhibited performances are hindering. According to the benchmark presented by [19], the proving time on the server could be 1000 times slower than the homomorphic computation itself. Since we already know

that homomorphic computation is slower than plaintext computation, this overhead makes QRPs impractical.

The second approach to proving directly over lattices is the work of LaBRADOR [20] and LatticeFold+ [21]. Here, LaBRADOR is a lattice-based SNARG and LatticeFold+ is a folding scheme. This approach achieves efficient proving time. For example, LaBRADOR has been applied in many constructions such as polynomial commitment scheme [22], aggregate signatures [23] and threshold ring signatures [24]. However, it still has several limitations.

- The constructions are usually efficient only for small rings, such as polynomial rings of dimension 64. However, most homomorphic encryption schemes require computations over large rings in order to bridge efficiency and security. This gap makes them less applicable for generating SNARG proofs for homomorphic encryption.
- Both LaBRADOR and LatticeFold+ only natively support non-NTT-friendly prime moduli. These are not typical choices for the majority of lattice-based protocols, as they make the arithmetic much slower.
- Their verification time is not optimal. For example, it is still unknown how to make the verification time of LaBRADOR sublinear. Thus, it does not satisfy the efficiency criterion that we are interested in.

High-level sketch of our solution. As previously discussed, the main challenge here is the mathematical gap between polynomial rings $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$ and finite fields. To address this gap, we adopt the ring-switching idea introduced by Huang, Mao and Zhang [25]. The core idea of ring switching is to replace the ring $\mathcal{R}_q$ over which the statement to prove is defined with an appropriately chosen extension field of $\mathbb{Z}_q$ of smaller size. At a high-level, ring switching proceeds in two steps:

- 1) Ring embedding. Firstly, an appropriate extension field $\mathbb{F}_q$ of $\mathbb{Z}_q$ is chosen. The size of $\mathbb{F}_q$ is related to the security parameter of the SNARG proof. For example, q is a 64-bit prime in our implementation, we then choose an extension degree of 2 to achieve 100-bit security. Once the extension field has been fixed, we then embed the statement $\mathbf{M}\mathbf{x} = \mathbf{y}$ over $\mathcal{R}_q$ into a statement over $\mathbb{F}_q[X]$.
- 2) Ring reduction. The second step of ring switching is to reduce the statement, via a polynomial commitment scheme over the extension field, from $\mathbb{F}_q[X]$ to $\mathbb{F}_q$. This ring reduction step runs in linear time, after which we can perform a standard SNARG proof over the extension field. We also note that the statement size becomes much smaller after ring reduction, since each $\mathcal{R}_q$ element is compressed into an $\mathbb{F}_q$ element.

2. Preliminary and Notations

Let $\mathbb{Z}$ denote the set of integers, and let $\mathbb{Z}_q$ denote the finite field of size q , where q is a prime. For a distribution D , we write $x \leftarrow D$ to denote sampling an element from D . We use $\mathcal{R}_q$ to denote the polynomial ring $\mathbb{Z}_q[X]/(X^N + 1)$.

2.1. GLWE and Additively Homomorphic Encryption

Definition 1. Let $N, \kappa, q \in \mathbb{Z}$ with N a power of 2 and χ_s and χ_e be two distributions over $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$. For $s \in \mathcal{R}_q^\kappa$, the *General LWE (GLWE) distribution* is defined as $(a, b = \langle a, s \rangle + e) \in \mathcal{R}_q^{\kappa+1}$, where $e \leftarrow \chi_e$ and $a \leftarrow \mathcal{U}(\mathcal{R}_q^\kappa)$. The *GLWE assumption* states that it is hard to distinguish the GLWE distribution from $\mathcal{U}(\mathcal{R}_q^{\kappa+1})$ when $s \leftarrow \chi_s$.

The hardness of the GLWE assumption with varying parameters can be estimated concretely using [26]. Let $p < q$ and $\Delta = \lfloor q/p \rfloor$. Based on the GLWE assumption, we can define the following secret key encryption scheme with message space $\mathcal{R}_p$, which goes back to [27].

Algorithm 1: A GLWE-based Encryption Scheme.
The parameters $N, \kappa, q, \Delta, \chi_s, \chi_e$ are public and available to all algorithms.

```

Gen :
  s ← χsκ
  return s
Encs(m) :
  a ← U(ℛqκ)
  e ← χe
  return c = (a, ⟨a, s⟩ + e + Δm)
Decs(c = (a, b)) :
  m ← ⌊(b - ⟨a, s⟩)/Δ⌋
  return m

```

The scheme in Algorithm 1 is secure based on the GLWE assumption. The scheme is correct, as long as $\|e/\Delta\|_\infty \leq 1/2$. It is easy to see that the scheme is additively homomorphic, i.e.

$$\text{Dec}_s \left(\sum_i \text{Enc}_s(m_i) \right) = \sum_i m_i$$

as long as parameters are such that $\|(\sum_i e_i)/\Delta\|_\infty \leq 1/2$. Ciphertexts can be multiplied with small polynomials, i.e.

$$\text{Dec}_s(m_1 \cdot \text{Enc}_s(m_2)) = m_1 \cdot m_2$$

as long as parameters are such that $\|(m_1 \cdot e_2)/\Delta\|_\infty \leq 1/2$.

2.2. Polynomial Commitment Scheme

Let us recall the definition of a Polynomial Commitment Scheme (PCS). A PCS over a field $\mathbb{F}$ is a quadruple of algorithms (Gen, Com, Eval, Verify) such that:

- $\text{Gen}(1^\lambda) \mapsto \text{pp}$ takes as input a security parameter and outputs public parameters pp.
- $\text{Com}(\text{pp}, f) \mapsto c$ takes as input the public parameters and a polynomial $f \in \mathbb{F}^{\leq d}[X]$ and outputs a commitment c to f .
- $\text{Eval}(\text{pp}, f, c, x, y) \mapsto \pi$ takes as input the public parameters pp, the polynomial f and its commitment c as

well as an evaluation point x and a claimed evaluation y of f at x . The algorithm outputs a proof π that $f(x) = y$.

- $\text{Verify}(\text{pp}, c, x, y, \pi) \mapsto \{\text{acc}/\text{rej}\}$ takes in the public parameters pp, a commitment c , an evaluation point x , a claimed evaluation y and an evaluation proof π and either accepts or rejects.

A PCS must observe the following properties:

- **Completeness.** Let $\text{pp} \leftarrow \text{Gen}(1^\lambda)$. For any polynomial $f \in \mathbb{F}^{\leq d}[X]$ and all domain elements x , let $\text{pp} \leftarrow \text{Gen}(1^\lambda)$, $c \leftarrow \text{Com}(\text{pp}, f)$ and $\pi \leftarrow \text{Eval}(\text{pp}, f, c, x, y)$. The Verify algorithm must accept with probability 1.
- **Evaluation binding.** Let $\text{pp} \leftarrow \text{Gen}(1^\lambda)$. For all f and for any $y \neq y'$ and π, π' , Verify accepts both $(\text{pp}, c, x, y, \pi)$ and $(\text{pp}, c, x, y', \pi')$ with negligible probability.

2.3. Succinct Non-Interactive Argument

Now we recall the definition of Succinct Non-Interactive Argument (SNARG).

Definition 2 (SNARG). A succinct non-interactive argument $\Pi = (\text{Gen}, \text{Prove}, \text{Verify})$ for an NP relation R is a tuple of algorithms with the following syntax.

- $\text{Gen}(1^\lambda) \mapsto \text{crs}$: Takes as input a security parameter $\lambda \in \mathbb{N}$ and outputs a common reference string crs.
- $\text{Prove}(\text{crs}, x, w) \mapsto \pi$: Takes as input common reference string crs, a statement x , and a witness w , and outputs a proof π when $(x, w) \in R$.
- $\text{Verify}(\text{crs}, x, \pi) \mapsto \{\text{acc}, \text{rej}\}$: Takes as input common reference string crs, a statement x , and a proof π , and outputs either acc (accept) or rej (reject).

The following security properties are required.

- **Completeness.** For all input $(x, w) \in R$, let $\text{crs} \leftarrow \text{Gen}(1^\lambda)$ and $\pi \leftarrow \text{Prove}(\text{crs}, x, w)$. The verifier accepts (crs, x, π) with probability 1.
- **Soundness.** For all $(x, w) \notin R$, and for any (crs, π) generated by a (possibly malicious) polynomial-time prover, the verifier accepts (crs, x, π) only with negligible probability.
- **Succinctness.** We say that the SNARG is succinct if the running time of Verify and the proof size are in $\text{poly}(\lambda, \log |x|, \log |w|)$.

2.4. Public Verifiable Homomorphic Matrix Multiplication

Now we explain our setting for publicly verifiable homomorphic matrix multiplication. It involves several steps.

1) Preprocessing stage:

- a) The client first performs some preprocessing for a publicly known matrix **A** and generates the encryption keys.

2) Computation stage:

- a) For a plaintext vector $\mathbf{x}$, the client encrypts it as $\mathbf{c}$ and sends $\mathbf{c}$ to the server.
- b) The server homomorphically computes $\mathbf{Ax}$ and returns $\mathbf{c}'$ together with a succinct proof π .
- c) The client verifies the proof and decrypts $\mathbf{c}'$.

Hereafter, we require several properties from the protocol:

- **Correctness.** If the server executes the whole protocol honestly, then $\text{Dec}(\mathbf{c}') = \mathbf{Ax}$ and the client accepts the proof π .
- **Soundness.** If the server deviates from the prescribed protocol, then the client rejects the proof with overwhelming probability.

Additionally, our goal for **succinctness** is to ensure that the client's online time, i.e. encryption, proof verification and decryption, is faster than computing $\mathbf{Ax}$ directly in the plaintext space.

3. Our Protocol

Now we give a formal description of our protocol in this section. Similar to standard vFHE, there are two parts in our protocol. The first part leverages a homomorphic encryption scheme to perform the computation over encrypted data. The second part is to generate a succinct proof to prove that the prescribed homomorphic computation has been correctly executed.

3.1. Homomorphic Computation

Let $\mathbf{A} \in \mathbb{Z}_p^{n \times m}$ and $\mathbf{x} \in \mathbb{Z}_p^m$. We now present a way to leverage the scheme from Algorithm 1 to perform the computation homomorphically on the encryption of $\mathbf{x}$. Crucially, the evaluation algorithm will only use arithmetic operations in $\mathcal{R}_q$ and thus be suitable to be combined with a SNARG for operations in $\mathcal{R}_q$, as demonstrated in Section 3.2.

The algorithm is a sub-algorithm from the one presented in [16], from which we strip all non-arithmetic operations to enable efficient verifiability. The key observation (already in [28]) is that for two polynomials $x = \sum_i x_i X^i \in \mathcal{R}_p$ and $y = \sum_i y_i X^i \in \mathcal{R}_p$ the constant term of $z = x \cdot y$ can be written as

$$\begin{aligned} z_0 &= x_0 \cdot y_0 - \sum_{i,j:i+j=N} x_i \cdot y_j \\ &= \langle \mathbf{x}, (y_0, -y_{N-1}, -y_{N-2}, \dots, -y_1) \rangle \end{aligned}$$

So let $\tau : \mathbb{Z}_p^N \mapsto \mathcal{R}_q$ be the encoding $\tau(\mathbf{y}) = y_0 - \sum_{i=1}^{N-1} y_{N-i} X^i$. Then $\langle \mathbf{y}, \mathbf{x} \rangle = (\tau(\mathbf{y}) \cdot x)_0$. Let $m > N$ and assume that $N \mid m$ (otherwise simply pad with zeros) and denote $t = m/N$. We extend $\tau : \mathbb{Z}_p^m \mapsto \mathcal{R}_p^t$ to vectors of size m by applying it to blocks of length N and concatenating the result into a vector of size t over $\mathcal{R}_p$. By also encoding the message $\mathbf{x} \in \mathbb{Z}_p^m$ into t elements $\mathbf{x} = (x_1, x_2, \dots, x_t) \in \mathcal{R}_p^t$ in the obvious way, we can extend the inner product computation to

$$\langle \mathbf{y}, \mathbf{x} \rangle = \langle \tau(\mathbf{y}), \mathbf{x} \rangle_0.$$

The algorithm now follows naturally and is given in Algorithm 2. We assume the user has already run Gen to generate a secret key $\mathbf{s}$.

Algorithm 2: An Algorithm for Homomorphic Matrix-Vector Multiplication

User on input $\mathbf{x} \in \mathbb{Z}_p^m$:
 $\mathbf{y} \leftarrow (\sum_{j=0}^N x_{i \cdot N + j} \cdot X^j)_{i=0}^{t-1} \in \mathcal{R}_q^t$
foreach $y_i \in \mathbf{y}$ **do**
 $\mathbf{c}_i \leftarrow \text{Enc}_{\mathbf{s}}(y_i)$
end
send $\mathbf{c} = (\mathbf{c}_i)_{i=0}^{t-1}$ to the server

Server on input $\mathbf{A} \in \mathbb{Z}_p^{n \times m}$ and ciphertext vector $\mathbf{c}$:
foreach row $\mathbf{a}_i \in \mathbb{Z}_p^m$ of $\mathbf{A}$ **do**
 $\mathbf{c}'_i \leftarrow \langle \tau(\mathbf{a}_i), \mathbf{c} \rangle$
end
send $\mathbf{c}' = (\mathbf{c}'_i)_{i=0}^{n-1}$ to the user

User on input $\mathbf{c}'$:
foreach $\mathbf{c}'_i$ **do**
 $z_i \leftarrow \text{Dec}_{\mathbf{s}}(\mathbf{c}'_i)_0$
end
return $\mathbf{z} = (z_1, \dots, z_n)$

Correctness. Let $\mathbf{c}'_i = (\mathbf{a}'_i, b'_i) \in \mathcal{R}_q^{\kappa+1}$ and $e'_i = b'_i - \langle \mathbf{a}'_i, \mathbf{s} \rangle \in \mathcal{R}_q$ be the corresponding noise polynomial. Note that we have $e'_i = \langle \tau(\mathbf{a}_i), \mathbf{e} \rangle$, where $\mathbf{e} \leftarrow \chi_e^t$ is the vector of noise polynomials in $\mathbf{c}$. For correctness, we require $\|e'_i\|_\infty = \|\langle \tau(\mathbf{a}_i), \mathbf{e} \rangle\|_\infty \leq \Delta/2$ for all i . Let σ^2 be the variance of χ_e . Then, the variance of each coordinate of $\langle \tau(\mathbf{a}_i), \mathbf{e} \rangle$ is $\leq m \cdot p^2 \cdot \sigma^2$. Heuristically assuming that each coordinate of $\langle \tau(\mathbf{a}_i), \mathbf{e} \rangle$ follows a Gaussian, we have that

$$\Pr[\|\langle \tau(\mathbf{a}_i), \mathbf{e} \rangle\|_\infty \geq r \cdot \sqrt{m} \cdot p \cdot \sigma] \leq \exp(-r^2/2)/\sqrt{2\pi r^2}$$

which is less than 2^{-130} for $r > 13.2$. So, if we set parameters such that $r \cdot \sqrt{m} \cdot p \cdot \sigma \leq \Delta/2$ for reasonable r , the protocol is correct with overwhelming probability. Equivalently, we require $q/\sigma \gtrsim 2r\sqrt{m}p^2$. We can use the lattice estimator [26] to find suitable k and N for any desired security level.

Optimizations. When sending many (G)LWE-based ciphertexts, a common technique to compress them is to not choose the $\mathbf{a}$ elements of the ciphertexts uniformly at random. Instead, a seed is chosen and the values $\mathbf{a}$ are all derived using a PRNG. Clearly, this is applicable here for the ciphertexts sent by the user (as pointed out in [16]).

Note that the result of the multiplication is stored in the constant coefficients of the plaintexts encrypted in $\mathbf{c}'_i$. So, the user does not need to decrypt the entire plaintext. Partially decrypting the constant coefficient can be done by first performing *sample extraction*, which is a technique that is folklore in the homomorphic encryption literature. Specifically, decryption requires computing $\langle \mathbf{a}, \mathbf{s} \rangle$, while all other operations are component-wise. It can be verified that

$\langle \mathbf{a}, \mathbf{s} \rangle_0 = \langle \tau^{-1}(\mathbf{a}), \bar{\mathbf{s}} \rangle$, where $\bar{\mathbf{s}} \in \mathbb{Z}_q^{\kappa N}$ is the coefficient vector of $\mathbf{s}$. Accordingly, the constant coefficient can be decrypted as $z_i \leftarrow \lfloor (b'_0 - \langle \tau^{-1}(\mathbf{a}'), \bar{\mathbf{s}} \rangle) / \Delta \rfloor$ where $\mathbf{c}'_i = (\mathbf{a}', b')$. This reduces decryption time from $O(\kappa N \log N)$ to $O(\kappa N)$ per coordinate of $\mathbf{z}$.

Performance. The user performs $t = m/N$ encryptions, each of cost $O(\kappa N \log N)$ for a total cost of $O(m \kappa \log N)$, and n decryptions for a total cost of $O(n \kappa N)$ (using the optimization described above). Thus, for $\kappa \log N = o(n)$ and $\kappa N = o(m)$, this yields a non-trivial solution for the user.

The server performs $m \cdot t \cdot \kappa$ polynomial multiplications in the ring $\mathcal{R}_q$ for a total cost of $O(m n \kappa \log N)$.

The user sends t ciphertexts containing N elements of $\mathbb{Z}_q$ each (using the compression technique outlined above), for a total number of m elements in $\mathbb{Z}_q$ and a short seed. The server sends n ciphertexts for a total number of $n \cdot (\kappa + 1) \cdot N$ element in $\mathbb{Z}_q$.

We note that some space is wasted when the server returns ciphertexts of which the client only needs to extract the constant coefficient. The work of [16] performs sample extraction on the server side and then re-pack the resulting LWE ciphertexts into RLWE ciphertexts, compressing them to n/N RLWE ciphertexts. However, this re-packing involves operations that are not purely arithmetic over $\mathcal{R}_q$ and thus are hard to reconcile with our SNARG for $\mathcal{R}_q$ in Section 3.2.

3.2. Efficient SNARG Proofs for Ring Operations

The Protocol. For simplicity, we describe the protocol for a matrix-vector multiplication of the form

$$\mathbf{y} \leftarrow \mathbf{M}\mathbf{x} \in \mathcal{R}_q^n$$

where $\mathbf{M}$ is the encoding of the matrix $\mathbf{A}$. We later show how to extend this to the setting described in Section 3.1 where the input to the multiplication is a vector of GLWE ciphertexts.

In order to prove integrity, the prover is essentially required to generate a proof for the statement $\mathbf{M}\mathbf{x} = \mathbf{y}$. In this regard, SNARGs appear to be a natural solution to this problem. However, as mentioned before, the main challenge is that $\mathbf{M}\mathbf{x} = \mathbf{y}$ is computed over the polynomial ring $\mathcal{R}_q$, which is usually large (thus involving more expensive arithmetic operations) and does not have a large exceptional set¹, preventing standard SNARG techniques from being applied.

We now explain how ring switching can address this challenge. The high-level idea of ring switching is to transfer the statement from $\mathcal{R}_q$ to an extension field of $\mathbb{Z}_q$, which we denote by $\mathbb{F}_q$. Ring switching consists of two steps. The first step lifts the statement over $\mathcal{R}_q$ to a statement over $\mathbb{F}_q[X]$. To achieve this, we first observe that $\mathbf{M}, \mathbf{x}, \mathbf{y}$ can be naturally considered as elements of $(\mathbb{Z}_q[X])^{n \times m}$, $(\mathbb{Z}_q[X])^m$, and $(\mathbb{Z}_q[X])^n$, respectively; that is, each entry of $\mathbf{M}, \mathbf{x}, \mathbf{y}$ is a polynomial in $\mathbb{Z}_q[X]$. Furthermore, we can assume that

the degree of each polynomial is at most $N - 1$, since $\mathcal{R}_q = \mathbb{Z}_q[X]/(X^N + 1)$. Based on this embedding, we know that $\mathbf{M}\mathbf{x} = \mathbf{y}$ over $\mathcal{R}_q$ if there exists a vector $\mathbf{r} \in \mathbb{Z}_q[X]$ with each entry being a polynomial of degree at most N such that

$$\mathbf{M}\mathbf{x} = \mathbf{y} + (X^N + 1) \cdot \mathbf{r}$$

Now we have converted the arithmetic structure from $\mathcal{R}_q$ to $\mathbb{Z}_q[X]$. Next, we choose an extension field of $\mathbb{Z}_q$, denoted by $\mathbb{F}_q$, and embed the statements into $\mathbb{F}_q[X]$. The purpose of this embedding is to enable the application of sumcheck protocols later. In lattice-based constructions, the native field $\mathbb{Z}_q$ is often too small to preserve soundness in sumcheck protocols. To conclude the ring embedding step, it is sufficient for the prover to prove that:

- $\mathbf{M}, \mathbf{x}, \mathbf{y}, \mathbf{r}$ are over $\mathbb{Z}_q[X]$, and
- $\mathbf{M}\mathbf{x} = \mathbf{y} + (X^N + 1) \cdot \mathbf{r}$ over $\mathbb{F}_q[X]$

Here, the prover has to additionally prove the first item in order to avoid soundness errors caused by the ring embedding. However, this part can actually be done by the verifier itself, since the verifier knows $\mathbf{M}, \mathbf{x}$, and $\mathbf{y}$. If the prover sends any $\mathbf{r}$ that is not in $\mathbb{Z}_q[X]$, then the second condition will not be satisfied. After this ring embedding step, we can use a multilinear extension to represent the statements, enabling a SNARG-style proof. Here, the multilinear extension is a commonly used step in constructing IPOP proofs; we refer the reader to [25] for details.

We recall that $\mathbf{M}$ is now a matrix in $(\mathbb{F}_q[X])^{n \times m}$. Hence, each entry $\mathbf{M}_{i,j}$ of $\mathbf{M}$ has the form,

$$\mathbf{M}_{i,j} = \sum_{k < N} (\mathbf{M}_{i,j})_k \cdot X^k$$

where $(\mathbf{M}_{i,j})_k$ are the coefficients of $\mathbf{M}_{i,j}$. Now let $\widetilde{\mathbf{M}}$ be the multilinear polynomial such that $\widetilde{\mathbf{M}}(i, j, k) = (\mathbf{M}_{i,j})_k$ for all $i \in [n], j \in [m]$ and $k \in [N]$. Here we slightly abuse the notation, as the variables of $\mathbf{M}$ actually represent the binary representation of i, j , and k . Similarly, we can define $\widetilde{\mathbf{x}}, \widetilde{\mathbf{y}}$, and $\widetilde{\mathbf{r}}$. Now it remains for the prover show that for every $i \in [n]$,

$$\begin{aligned} & \sum_j \left(\sum_k \widetilde{\mathbf{M}}(i, j, k) X^k \right) \left(\sum_k \widetilde{\mathbf{x}}(j, k) X^k \right) \\ &= \sum_k \widetilde{\mathbf{y}}(i, k) X^k + \sum_k \widetilde{\mathbf{r}}(i, k) X^k \cdot (X^N + 1) \end{aligned}$$

For each i , we define a degree- $2(N - 1)$ univariate polynomial $F_i(X)$ over $\mathbb{F}_q$ by

$$\begin{aligned} F_i(X) := & \sum_j \left(\sum_k \widetilde{\mathbf{M}}(i, j, k) X^k \right) \left(\sum_k \widetilde{\mathbf{x}}(j, k) X^k \right) \\ & - \sum_k \widetilde{\mathbf{y}}(i, k) X^k + \sum_k \widetilde{\mathbf{r}}(i, k) X^k \cdot (X^N + 1) \end{aligned}$$

Then it is essentially equivalent to proving that $F_i(X) \equiv 0$. Here, we can use a standard polynomial zero test: the

1. Large exceptional sets are necessary for soundness requirements.

verifier randomly sends a point $\alpha \in \mathbb{F}_q$, and the prover then proves that for all $i \in [n]$, $F_i(\alpha) = 0$. The soundness error is upper bounded by $2N/|\mathbb{F}_q| = O(N/\mathbb{F}_q)$. Notice that there are n constraints here, as the prover needs to prove that $F_i(\alpha) = 0$ for all $i \in [n]$. We then use a standard batching technique to combine them. Define a $(\log n)$ -variable polynomial $f(t)$ to batch all the constraints.

$$f(t) = \sum_i \widetilde{\mathbf{eq}}(t, i) F_i(\alpha)$$

This is the unique multilinear polynomial that encodes the n evaluations of $F_i(\alpha)$. As such, in order to prove that $F_i(\alpha) = 0$ for all $i \in [n]$, it suffices to prove that $f(t)$ is the zero polynomial. Note that $f(t)$ is a polynomial over $\mathbb{F}_q$, and the ring-switching process has now been completed. By the Schwartz-Zippel lemma, for a random $\tau \xleftarrow{\$} \mathbb{F}_q^{\log n}$, $f(\tau) = 0$ implies with high probability that $f(t)$ is the zero polynomial. The verifier can thus sample such a random τ , send it to the prover who can then use the standard sumcheck protocol over the finite field to prove that

$$f(\tau) = 0. \quad (1)$$

To do so, we first expand the components of $f(\tau)$. We define $\widetilde{\ell}(\mathbf{x})$ to be the multilinear extension of α^k for all $k \in [N]$. Let

$$\begin{aligned} \zeta_1 &:= \sum_i \widetilde{\mathbf{eq}}(\tau, i) \sum_j \left(\sum_k \widetilde{\mathbf{M}}(i, j, k) \widetilde{\ell}(k) \right) \widetilde{\mathbf{x}}_\alpha(j) \\ \zeta_2 &:= \sum_i \widetilde{\mathbf{eq}}(\tau, i) \sum_k \widetilde{\mathbf{y}}_\alpha(i) \\ \zeta_3 &:= \sum_i \widetilde{\mathbf{eq}}(\tau, i) \sum_k \widetilde{\mathbf{r}}(i, k) \widetilde{\ell}(k) \end{aligned}$$

The prover computes ζ_1 and ζ_3 and sends these values to the verifier, who computes ζ_2 by itself and checks that

$$\zeta_1 - \zeta_2 - \zeta_3 \cdot (\alpha^N + 1) = 0$$

which implies that Equation (1) holds, provided the values sent by the prover are well-formed. Let

$$\begin{aligned} g(i, j, k) &:= \widetilde{\mathbf{eq}}(\tau, i) \cdot \widetilde{\mathbf{M}}(i, j, k) \cdot \widetilde{\mathbf{x}}_\alpha(j) \cdot \widetilde{\ell}(k) \\ h(i, k) &:= \widetilde{\mathbf{eq}}(\tau, i) \cdot \widetilde{\mathbf{r}}(i, k) \cdot \widetilde{\ell}(k) \end{aligned}$$

To convince the verifier that ζ_1 and ζ_3 are well-formed, the prover and verifier run the sumcheck protocol twice, once for

$$\sum_{i,j,k} g(i, j, k) = \zeta_1 \quad (2)$$

and once for

$$\sum_{i,k} h(i, k) = \zeta_3. \quad (3)$$

The sumcheck protocol reduces the validity of Equations (2) and (3) to checking that $g(\rho_1) = v_1$ and $h(\rho_2) = v_2$ where ρ_1, ρ_2 are vectors of random values sampled from the sum-check challenge space by the verifier throughout the

execution of the sumcheck protocol and v_1, v_2 are values determined by the execution of the protocol. For ζ_1 , the verifier can perform this check efficiently by computing by itself

$$\begin{aligned} \nu_1 &:= \widetilde{\mathbf{eq}}(\tau, \rho_1[\dots \log n]) \\ \nu_2 &:= \widetilde{\mathbf{x}}_\alpha(\rho_1[\log n \dots \log nm]) \\ \nu_3 &:= \widetilde{\ell}(\rho_1[\log nm \dots]) \end{aligned}$$

and checking that

$$v_1 = \nu_1 \cdot \mu \cdot \nu_2 \cdot \nu_3$$

holds, where μ denotes the evaluation of $\widetilde{\mathbf{M}}(\rho_1)$ provided by the prover using a polynomial commitment scheme. If this last check holds, then the verifier is convinced that the value ζ_1 provided by the prover is well-formed. The prover and verifier proceed analogously to ensure that Equation (3) holds, this time with $\widetilde{\mathbf{r}}(\rho_2)$ being provided by the PCS.

It is worth noting that the value ζ_2 is not subject to this technique because the verifier needs to know $\mathbf{y}$ in full for decryption. This is not the case for $\mathbf{r}$, so the overall protocol proof size benefits from including only a commitment and evaluation proof for $\widetilde{\mathbf{r}}$, instead of the full vector $\mathbf{r}$. The verifier can compute ζ_2 efficiently by itself.

Algorithm 3: An Efficient SNARG over Rings

Server and **User** preprocess the matrix $\mathbf{M}$;
Server commits to the polynomial $\widetilde{\mathbf{M}}$;

User sends $\mathbf{x}$ to the server;

Server sends $\mathbf{y}$ and a commitment to $\widetilde{\mathbf{r}}$ to the user;

Server and **User** run the sumcheck protocol to prove that $f(\tau) = 0$;

User makes one query to $\widetilde{\mathbf{M}}$ and one query to $\widetilde{\mathbf{r}}$ and accepts or rejects.

Optimizations. The above protocol is defined for a matrix-vector product $\mathbf{y} = \mathbf{M}\mathbf{x}$ over $\mathcal{R}_q$. However, the homomorphic matrix-vector multiplication algorithm defined in Algorithm 2 multiplies a plaintext matrix with a vector of GLWE ciphertexts encrypting $\mathbf{x}$. As described in Section 2.1, such ciphertexts are vectors of dimension $\kappa + 1$ over $\mathcal{R}_q$. As such, the protocol described above only allows us to prove the correct execution of matrix-vector multiplication for a *single* component of the resulting GLWE ciphertexts. A naive way to extend the above protocol is to simply repeat it $\kappa + 1$ times, once for each component of the GLWE ciphertexts. A more efficient manner of extending the protocol to the desired functionality is to require the

verifier to sample a supplementary random challenge β and have the prover and verifier run the above protocol for

$$\sum_j \left(\sum_k \widetilde{\mathbf{M}}(i, j, k) X^k \right) \left(\sum_{\iota=1}^{\kappa+1} \beta^\iota \left(\sum_k \widetilde{\mathbf{x}}_\iota(j, k) X^k \right) \right) \\ = \sum_{\iota=1}^{\kappa+1} \beta^\iota \left(\sum_k \widetilde{\mathbf{y}}_\iota(i, k) X^k + \sum_k \widetilde{\mathbf{r}}_\iota(i, k) X^k \cdot (X^N + 1) \right)$$

for all $i \in [n]$.

Analysis of the SNARG Proof. We now analyze both the running time and the security of the SNARG protocol.

The prover’s running time consists of a PIOP (polynomial interactive oracle proof) and a polynomial commitment evaluation. The PIOP component mainly involves a sumcheck protocol, which runs in linear time, while the polynomial commitment incurs a quasi-linear time cost. Note that the matrix has size $O(nm)$, the PCS running time is $O(nm \log(nm))$. For the verification part, both the PIOP and the PCS have logarithmic running time. Therefore,

- The prover’s running time is $O(nm \log(nm))$;
- The verifier’s running time is $O(\log(nm))$;
- The proof size is also $O(\log(nm))$.

We note that both our PIOP and WHIR protocols have perfect completeness, and hence the SNARG also has perfect completeness. For soundness, the PIOP mainly relies on polynomial testing, which by the Schwartz-Zippel lemma yields a soundness error of

$$O(N/|\mathbb{F}_q| + \log(n)/|\mathbb{F}_q| + \kappa/|\mathbb{F}_q|).$$

Overall, by choosing a proper extension field size, we can achieve a negligible soundness error. In our implementation, we choose q to be a 64-bit prime and $\mathbb{F}_q$ is a degree-2 extension field. Overall, this gives 100-bit security.

4. Experimental analysis

We implement the above protocol in Rust to measure its concrete performance and open-source the implementation². Our prototype confirms the hypothesis of Section 3.1 according to which our overall protocol’s verifier is concretely faster than the standard $O(mn)$ matrix-vector multiplication algorithm over $\mathbb{Z}_p$.

4.1. Implementation description

Our prototype implements the protocol as described Section 3.2, including all optimizations discussed in this work. We render the protocol non-interactive via the Fiat-Shamir transform using BLAKE3 as the hash function [29]. The protocol relies on a polynomial commitment scheme in a black-box manner. In other words, it can be instantiated with any PCS. For performance and ease of use purposes, we choose in our implementation to instantiate the PCS

². The code is available at:
<https://github.com/tremblaythibault/matvecmul>

TABLE 1. PARAMETERS

q	N	κ	$\log_2 \sigma$	p
$2^{64} - 2^{32} + 1$	1024	1	33	16

with the WHIR polynomial commitment scheme [30] set to target 100 bits of security. Naturally, this means that our implementation directly inherits WHIR’s performance. It is worth noting that our protocol can readily be adapted to forthcoming polynomial commitment schemes that perform better than WHIR.

We use the parameters from Table 1 which allow us to claim approximately 100 bits of security for the encryption scheme given in Section 2.1 according to the well-known Lattice Estimator [31], all the while maintaining correctness of the results. Letting $\mathbb{Z}_q$ be our base prime field and $\mathbb{F}_q$ be the quadratic extension field of $\mathbb{Z}_q$, we are able to claim 100 bits of security for the SNARG according to the discussion in the previous section.

4.2. Experiments

For each experiment, we run the process at least 10 times and take the median measurement as the representative. The machine used to run the experiments is an AWS EC2 instance of type `Hpc7a.24xlarge` equipped with 48 CPU cores and 768 GB of memory.

Prover time. Table 2 provides the prover time as a function of the matrix parameters $t := m/N$ and n . The prover times are measured both with and without the PCS computation in order to clearly present the performance of our SNARG. The time taken by the prover without the PCS enabled is nearly always less than half that taken by the prover with the PCS enabled. This indicates that our protocol’s prover time is actually dominated by the PCS prover time. Hence, given a PCS exhibiting better performances than WHIR, we can expect our protocol’s online computation time to significantly decrease.

Verifier time. The overall protocol verifier time is taken as the sum of:

- Time taken to encrypt $\mathbf{x}$;
- Time taken to verify the SNARG proof;
- Time taken to decrypt $\mathbf{c}'$.

We call this value “compiled verifier time”. Figures 1 to 3 provide comparisons between the compiled verifier time and the baseline $O(mn)$ plaintext matrix-vector multiplication algorithm over $\mathbb{Z}_p$, again for matrix parameters $t := m/N$ and n . There is no clear bottleneck for the verifier time, as different steps contribute differently to the compiled verifier time depending on the parameters. Our experiments show that our verifier starts to outperform the baseline $O(mn)$ plaintext matrix-vector multiplication algorithm when $m = 2^{17}$ and $n = 2^8$. The SNARG prover only takes a few seconds to execute in this parameter regime, which demonstrates the practicality of our protocol.

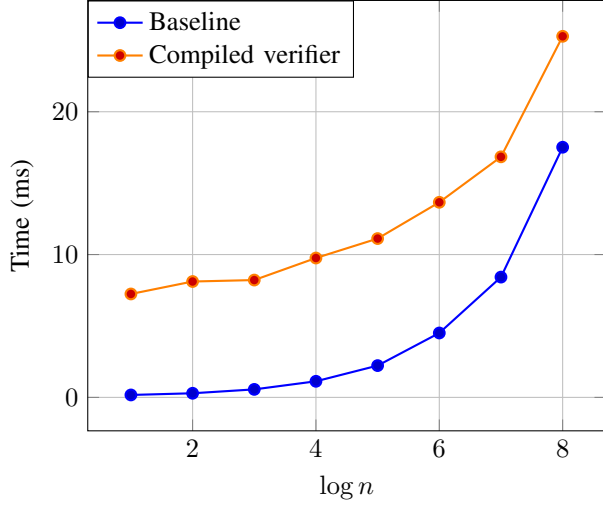


Figure 1. Timing for $t = 2^6$ across n values.

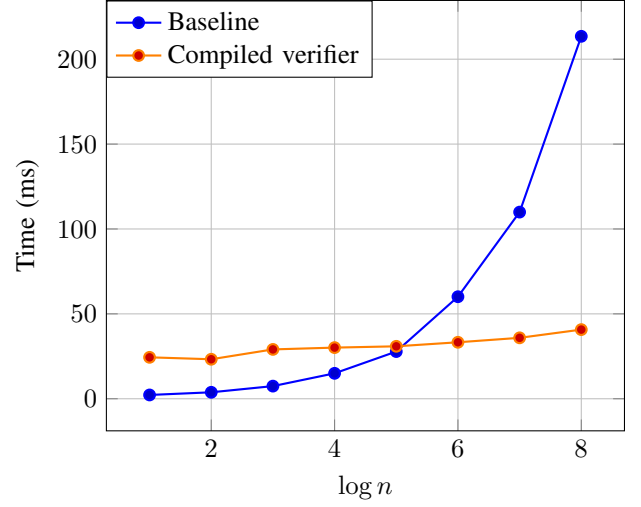


Figure 3. Timing for $t = 2^{10}$ across n values.

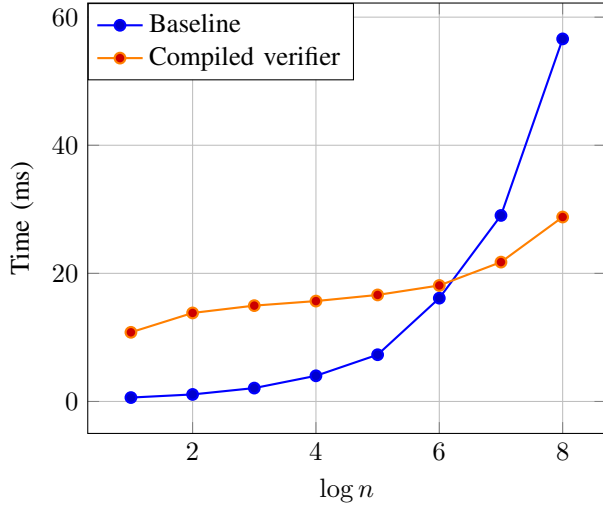


Figure 2. Timing for $t = 2^8$ across n values.

Communication costs. The SNARG proof size is taken as the sum of:

- The sumcheck proof sizes for the ζ_1 and ζ_3 claims;
- The WHIR evaluation proof sizes for $\widetilde{\mathbf{M}}(\rho_1)$ and $\widetilde{\mathbf{r}}(\rho_2)$.

Table 3 presents the proof sizes as a function of t and n . Our results exhibit the very succinct proof sizes resulting from our protocol. Furthermore, the WHIR evaluation proofs account for more than 97% of the total proof size. Given a PCS with shorter proofs, one can expect the proof sizes of our overall protocol to decrease drastically.

Recent work [32], [33] suggests that conjectures invoked for performance purposes in WHIR may not hold. If these results are correct, WHIR proof sizes may be subject to a $\approx 4\times$ blowup.

In addition to the SNARG proof, the prover also sends the ciphertext vector $\mathbf{c}'$ resulting from the protocol (c.f. Algorithm 2). This vector has bit size $n(\kappa+1)N \log q$, which

amounts to between 32 kB for our smallest experiments and 4 MB for our largest experiments. This measure grows linearly in the number of rows of $\mathbf{A}$ and is represented in Table 4. After decryption, the output vector ($\mathbf{z}$ in Algorithm 2) has size $n \log p$, which ranges between 1 B and 128 B.

The plaintext input vector ($\mathbf{x}$ in Algorithm 2) has bit size $m \log p$, which ranges from 512 B for our smaller experiments to 512 kB for our larger experiments. Encrypting this vector using the compression technique discussed in Section 3.1 yields an input ciphertext vector ($\mathbf{c}$ in Algorithm 2) of size $tN \log q$, which ranges from 16 kB to 8 MB. This measure grows linearly with t the number of columns in the matrix $\mathbf{M}$ and is given in Table 5. The compression method follows prior work [16] and uses a 64-bit seed.

Acknowledgements. The authors would like to thank Petar Ivanov for insightful discussions and help in optimizing the implementation.

References

- [1] D. Fiore and R. Gennaro, “Publicly verifiable delegation of large polynomials and matrix computations, with applications,” in *CCS*. ACM, 2012, pp. 501–512.
- [2] L. de Castro and K. Lee, “VeriSimplePIR: Verifiability in SimplePIR at no online cost for honest servers,” in *USENIX Security 2024: 33rd USENIX Security Symposium*, D. Balzarotti and W. Xu, Eds. Philadelphia, PA, USA: USENIX Association, August 14–16, 2024. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/de-castro>
- [3] L. T. Thibault and M. Walter, “Towards verifiable FHE in practice: Proving correct execution of tfhe’s bootstrapping using plonky2,” in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS 2025, Taipei, Taiwan, October 13–17, 2025*, C. Huang, J. Chen, S. Shieh, D. Lie, and V. Cortier, Eds. ACM, 2025, pp. 1113–1126. [Online]. Available: <https://doi.org/10.1145/3719027.3765097>

- [4] S. Atapoor, K. Bagheri, H. V. L. Pereira, and J. Spiessens, "Verifiable FHE via lattice-based snarks," *IACR Commun. Cryptol.*, vol. 1, no. 1, p. 24, 2024.
- [5] M.-Y. M. Huang, B. Li, X. Mao, and J. Zhang, "Fully homomorphic encryption with efficient public verification," *Cryptology ePrint Archive*, 2024.
- [6] I. Cascudo, A. Costache, D. Cozzo, D. Fiore, A. Guimarães, and E. Soria-Vazquez, "Verifiable computation for approximate homomorphic encryption schemes," in *CRYPTO (7)*, ser. Lecture Notes in Computer Science, vol. 16006. Springer, 2025, pp. 643–677.
- [7] F. Liu, H. Liang, T. Zhang, Y. Hu, X. Xie, H. Tan, and Y. Yu, "Hasteboots: Proving FHE bootstrapping in seconds," *IACR Cryptol. ePrint Arch.*, p. 261, 2025.
- [8] Z. Zhou, Y. Li, Y. Wang, Z. Yang, B. Zhang, C. Hong, T. Wei, and W. Chen, "Zhe: Efficient zero-knowledge proofs for he evaluations," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 3328–3346.
- [9] F. Liu, H. Liang, T. Zhang, Y. Hu, X. Xie, H. Tan, and Y. Yu, "Hasteboots: Proving fhe bootstrapping in seconds," *Cryptology ePrint Archive*, 2025.
- [10] C. Knabenhans, A. Viand, A. Merino-Gallardo, and A. Hithnawi, "vfhe: Verifiable fully homomorphic encryption," in *WAHC@CCS*. ACM, 2024, pp. 11–22.
- [11] W.-j. Lu and J. Sakuma, "More practical privacy-preserving machine learning as a service via efficient secure matrix multiplication," in *Proceedings of the 6th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, 2018, pp. 25–36.
- [12] J. Liu and L. F. Zhang, "Privacy-preserving and publicly verifiable matrix multiplication," *IEEE Transactions on Services Computing*, vol. 16, no. 3, pp. 2059–2071, 2022.
- [13] Y. Gao, G. Quan, S. Homsi, W. Wen, and L. Wang, "Secure and efficient general matrix multiplication on cloud using homomorphic encryption," *The Journal of Supercomputing*, vol. 80, no. 18, pp. 26 394–26 434, 2024.
- [14] X. Ma, C. Ma, Y. Jiang, and C. Ge, "Improved privacy-preserving pca using optimized homomorphic matrix multiplication," *Computers & Security*, vol. 138, p. 103658, 2024.
- [15] H. Mahon and S. Kosieradzki, "Encrypted matrix multiplication using 3-dimensional rotations," *Cryptology ePrint Archive*, 2025.
- [16] J. Frery, R. Bredehoft, J. Klemsa, A. Meyre, and A. Stoian, "Private lora fine-tuning of open-source llms with homomorphic encryption," 2025. [Online]. Available: <https://arxiv.org/abs/2505.07329>
- [17] A. Akavia, C. Gentry, S. Halevi, and M. Vald, "Achievable CCA2 relaxation for homomorphic encryption," in *TCC (2)*, ser. Lecture Notes in Computer Science, vol. 13748. Springer, 2022, pp. 70–99.
- [18] C. Ganesh, A. Nitulescu, and E. Soria-Vazquez, "Rinocchio: Snarks for ring arithmetic," *Journal of Cryptology*, vol. 36, no. 4, p. 41, 2023.
- [19] M. H. Santriaji, J. Xue, Y. Zhang, Q. Lou, and Y. Solihin, "Dataseal: Ensuring the verifiability of private computation on encrypted data," in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 2378–2394.
- [20] W. Beullens and G. Seiler, "Labrador: compact proofs for rlcs from module-sis," in *Annual International Cryptology Conference*. Springer, 2023, pp. 518–548.
- [21] D. Boneh and B. Chen, "Latticefold+: faster, simpler, shorter lattice-based folding for succinct proof systems," in *Annual International Cryptology Conference*. Springer, 2025, pp. 327–361.
- [22] N. K. Nguyen and G. Seiler, "Greyhound: fast polynomial commitments from lattices," in *Annual International Cryptology Conference*. Springer, 2024, pp. 243–275.
- [23] M. A. Aardal, D. F. Aranha, K. Boudgoust, S. Kolby, and A. Takahashi, "Aggregating falcon signatures with labrador," in *Annual International Cryptology Conference*. Springer, 2024, pp. 71–106.
- [24] S. Jeon, C. Abou Haidar, and M. Tibouchi, "Lastrings: Lattice-based scalable threshold ring signatures," *Cryptology ePrint Archive*, 2025.
- [25] M. M. Huang, X. Mao, and J. Zhang, "Sublinear proofs over polynomial rings," *IACR Cryptol. ePrint Arch.*, p. 199, 2025.
- [26] M. R. Albrecht, R. Player, and S. Scott, "On the concrete hardness of learning with errors," *J. Math. Cryptol.*, vol. 9, no. 3, pp. 169–203, 2015.
- [27] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," in *STOC*. ACM, 2005, pp. 84–93.
- [28] Z. Huang, W. Lu, C. Hong, and J. Ding, "Cheetah: Lean and fast secure two-party deep neural network inference," in *USENIX Security Symposium*. USENIX Association, 2022, pp. 809–826.
- [29] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn, "Blake3: One function, fast everywhere," 2021. [Online]. Available: <https://github.com/BLAKE3-team/BLAKE3-specs/blob/master/blake3.pdf>
- [30] G. Arnon, A. Chiesa, G. Fenzi, and E. Yogeve, "WHIR: Reed-solomon proximity testing with super-fast verification," in *Advances in Cryptology – EUROCRYPT 2025, Part IV*, ser. Lecture Notes in Computer Science, S. Fehr and P.-A. Fouque, Eds., vol. 15604. Madrid, Spain: Springer, Cham, Switzerland, May 4–8, 2025, pp. 214–243.
- [31] M. R. Albrecht, R. Player, and S. Scott, "On the concrete hardness of learning with errors," *J. Math. Cryptol.*, vol. 9, no. 3, pp. 169–203, 2015. [Online]. Available: <http://www.degruyter.com/view/j/jmc.2015.9.issue-3/jmc-2015-0016/jmc-2015-0016.xml>
- [32] B. E. Diamond and A. Gruen, "On the distribution of the distances of random words," *Cryptology ePrint Archive*, Paper 2025/2010, 2025. [Online]. Available: <https://eprint.iacr.org/2025/2010>
- [33] E. Crites and A. Stewart, "On reed-solomon proximity gaps conjectures," *Cryptology ePrint Archive*, Paper 2025/2046, 2025. [Online]. Available: <https://eprint.iacr.org/2025/2046>

TABLE 2. PROVER TIMES (IN SECONDS). THE ROWS WITH A ✓ INDICATE MEASUREMENTS WITH THE PCS ENABLED, WHILE THE ROWS WITH A ✗ INDICATE MEASUREMENTS WITHOUT THE PCS.

$\log n \backslash \log t$		1	2	3	4	5	6	7	8	9	10
1	✓	0.05	0.06	0.07	0.09	0.12	0.17	0.25	0.39	0.60	1.10
	✗	0.01	0.02	0.02	0.03	0.04	0.07	0.10	0.17	0.30	0.56
2	✓	0.07	0.08	0.10	0.13	0.18	0.25	0.37	0.57	0.98	1.56
	✗	0.02	0.02	0.03	0.04	0.06	0.09	0.14	0.24	0.43	0.81
3	✓	0.09	0.10	0.14	0.19	0.24	0.35	0.54	0.96	1.44	2.70
	✗	0.02	0.03	0.04	0.06	0.09	0.13	0.21	0.38	0.68	1.31
4	✓	0.12	0.15	0.21	0.25	0.37	0.54	0.98	1.80	2.89	4.63
	✗	0.03	0.04	0.06	0.09	0.13	0.21	0.35	0.64	1.23	2.41
5	✓	0.17	0.22	0.29	0.40	0.58	1.11	1.78	3.81	5.87	12.47
	✗	0.05	0.06	0.09	0.13	0.20	0.34	0.61	1.20	2.26	4.49
6	✓	0.28	0.33	0.48	0.67	1.20	1.97	3.81	6.69	13.72	24.80
	✗	0.07	0.09	0.13	0.21	0.35	0.62	1.15	2.31	4.47	9.62
7	✓	0.43	0.56	0.79	1.30	2.05	3.92	6.74	13.91	25.04	53.37
	✗	0.11	0.16	0.23	0.36	0.63	1.19	2.31	4.54	9.41	22.64
8	✓	0.66	0.88	1.42	2.14	4.04	6.83	13.94	24.96	53.22	101.55
	✗	0.17	0.24	0.38	0.63	1.16	2.23	4.56	9.40	22.65	45.06

TABLE 3. PROOF SIZES (IN KB).

$\log n \backslash \log t$	1	2	3	4	5	6	7	8	9	10
1	130	135	138	145	148	154	155	156	165	161
2	158	157	162	166	171	175	178	183	184	188
3	167	172	178	183	181	187	192	194	197	198
4	176	184	189	190	192	199	201	204	206	210
5	190	195	197	200	206	207	211	213	217	220
6	207	210	211	219	220	223	225	230	231	234
7	215	219	223	227	228	232	235	239	239	242
8	224	228	231	233	236	240	241	244	247	250

TABLE 4. OUTPUT CIPHERTEXT VECTOR SIZE, USING PARAMETERS FROM TABLE 1 (IN KB).

$\log n$	1	2	3	4	5	6	7	8
$ \mathbf{c}' $	32	64	128	256	512	1024	2048	4096

TABLE 5. INPUT COMPRESSED CIPHERTEXT VECTOR SIZE, USING PARAMETERS FROM TABLE 1 (IN KB).

$\log t$	1	2	3	4	5	6	7	8	9	10
$ \mathbf{c} $	16	32	64	128	256	512	1024	2048	4096	8192